

Deep Learning — Complete Study Notes

Easy explanations, simple words, real analogies — everything explained clearly.

What's inside

- What is Deep Learning & why it's popular now
- Neural Networks — the basic building block
- How Neural Networks learn (Forward & Back Propagation)
- Core terms: Activation Functions, Loss Functions, Optimizers
- Parameters vs Hyperparameters, Epochs/Batches/Iterations
- Types of Learning: Supervised, Unsupervised, Reinforcement
- Overfitting & Regularization (Dropout, Data Augmentation, Early Stopping)
- Neural Network Architectures: Feedforward, RNN, CNN
- The 5 Steps of Every Deep Learning Project

1. What is Deep Learning?

Deep Learning is a part of Machine Learning, and Machine Learning is a part of Artificial Intelligence (AI). Think of it like nested boxes:

🔍 Simple analogy: AI is the biggest box (any machine trying to act smart). Inside it is Machine Learning (machines that learn from data instead of being told exact rules). Inside that is Deep Learning (machine learning done using layered 'neural networks' inspired by the brain).

In simple words: Deep Learning is a technique where a computer learns patterns and tasks directly from raw data (like images, text, or sound) by passing that data through a network of layers, without a human manually telling it what features to look for.

Why is it called 'deep'?

Because the neural network has many hidden layers stacked between the input and output. Data goes 'deep' through these layers, and at each layer the network builds a more complex understanding — first simple patterns (like edges in an image), then more complex ones (like eyes, then a full face).

Famous moments that made Deep Learning famous

- 1997 — IBM's Deep Blue defeated world chess champion Garry Kasparov.
- 2011 — IBM's Watson won Jeopardy against two human champions.
- 2015/2016 — Google DeepMind's AlphaGo defeated Go champion Lee Sedol, in a game far more complex than chess.

Today deep learning is used in self-driving cars, cancer diagnosis, language translation, fake news detection, and more.

Why is Deep Learning popular only now (not decades ago)?

The math behind neural networks has existed for a long time. Three things changed recently:

- More Data — we now generate huge amounts of data ("big data") that these models need.
- Better Hardware — powerful GPUs can now handle the heavy calculations.
- Better Tools — free, easy-to-use software like TensorFlow and PyTorch made building models much simpler.

Why not just use traditional Machine Learning?

Traditional ML needs a human to manually tell the computer what 'features' to look for (for example: 'a face has eyes, a nose, a mouth, and eyes have corners at a certain angle'). This is very hard to do by hand for complex data.

🔍 Simple analogy: Imagine trying to explain to someone over the phone, using only words, exactly what a human face looks like — every angle, every curve. Very hard! Deep learning skips this — you just show it thousands of face photos, and it figures out the patterns itself.

Deep learning removes this manual work. You just feed it raw data (like many photos), and the network automatically learns to detect simple patterns first (lines, edges), then combines them into more complex ones (eyes, nose), and finally recognizes the whole object (a face).

2. Neural Networks — The Basic Building Block

A Neural Network is inspired by how neurons work in the human brain. It is made of small units called neurons, arranged in layers.

Every neural network has 3 main parts:

- Input Layer — where the raw data enters (e.g., pixels of an image).
- Hidden Layer(s) — the layers in between, where the actual 'thinking'/pattern-finding happens. There can be many of these.
- Output Layer — gives the final result (e.g., 'this is a cat', or 'this is a dog').

 *The network takes in data, tries to recognise patterns in it, and then uses those patterns to make predictions on new, similar data it hasn't seen before.*

3. How Do Neural Networks Learn?

Learning happens in two main steps that repeat over and over: Forward Propagation and Back Propagation.

Step 1: Forward Propagation (making a guess)

Information flows from the Input layer → through Hidden layers → to the Output layer. Here's what happens at each connection:

- Every input is multiplied by a 'weight' (a number showing how important that input is).
- All these weighted inputs are added together, and a 'bias' number is added too (bias just shifts the result up or down, giving the neuron a bit of its own 'opinion').
- This total is passed through an 'Activation Function', which decides how much this neuron should 'fire' or contribute to the next layer.
- In the output layer, this whole process gives a set of probabilities — the neuron with the highest value becomes the final prediction.

Step 2: Back Propagation (learning from mistakes)

After a prediction is made, the network checks: was it right or wrong? It measures how wrong it was using a Loss Function (explained below).

Then, this error information flows backward — from the output layer back through the hidden layers — and the network slightly adjusts its weights and biases so that next time, it makes a better guess.

 **Simple analogy:** Think of it like practicing a video game. You play a level (forward propagation = making a guess), you see your score/mistakes (loss function), and next time you adjust your strategy slightly (back propagation) to do better. You repeat this many, many times.

Worked example: Cars vs Trucks

Say we have data about vehicles: their weight and how many goods they carry, and whether they are a 'car' or 'truck'.

- Start: give the network completely random weights and biases (it knows nothing yet).
- Feed it one example, e.g., weight = 15, goods = 2, and the network makes a guess (say it wrongly guesses 'truck' when the answer is 'car').
- Compare the guess to the correct answer using a loss function → this tells us how wrong it was.
- Use back propagation to adjust the weights and biases slightly, so it does a bit better next time.
- Repeat this for every example in the dataset. Over time, with enough repetition, the network gets good at telling cars from trucks.

💡 *More data generally helps the network learn better — but too much repetition on the same data can cause 'overfitting' (explained later).*

The learning process, summarised in 5 steps

1. Start with random weights and biases.
2. Pass input data through the network (forward propagation) to get a prediction.
3. Compare the prediction to the correct answer using a Loss Function.
4. Send this error backward through the network (back propagation) to see how each weight contributed to the mistake.
5. Update the weights and biases using Gradient Descent to reduce the error. Repeat steps 2–5 until the model is 'good enough'.

4. Core Terminologies in Deep Learning

A) Activation Functions

An activation function decides whether/how much a neuron should 'fire' (contribute) to the next layer. It also adds 'non-linearity', which lets the network learn complex patterns instead of just straight-line relationships.

💡 Without non-linear activation functions, stacking many layers would be pointless — the whole deep network would behave like just ONE simple layer, no matter how many layers you add.

Function	What it does	Pros	Cons
Step Function	Outputs 1 if input > threshold, else 0 (binary on/off)	Very simple	Can't handle multiple classes well; not smooth
Linear Function	Output is directly proportional to input (straight line)	Simple, gives a range of values	Stacking layers becomes pointless (whole network = 1 layer); gradient is constant, not useful for learning
Sigmoid	Squeezes any input into a range between 0 and 1	Non-linear (can stack layers); smooth output	Vanishing gradient problem — barely updates for very large/small inputs
Tanh	Like Sigmoid but ranges from -1 to 1	Non-linear, steeper gradient than Sigmoid	Still has vanishing gradient problem
ReLU	Outputs 0 for negative input, and the input itself if positive	Cheap to compute; creates 'sparse' (efficient) activation	'Dying ReLU' problem — some neurons can get stuck outputting 0 forever
Leaky ReLU	Like ReLU, but allows a small slope for negative inputs instead of flat 0	Fixes the dying ReLU problem	One extra small parameter to manage

💡 General tip: if you're not sure which activation function to use, start with ReLU — it's the most commonly used default today. Use Sigmoid for binary (yes/no) classification output layers.

B) Loss Functions

A Loss Function simply measures how wrong the network's prediction was compared to the actual correct answer. That's it — a way to put a number on 'how bad was this guess'.

Problem Type	Common Loss Functions
Regression (predicting numbers)	Squared Error Loss, Absolute Error Loss, Huber Loss
Binary Classification (2 classes)	Binary Cross-Entropy, Hinge Loss
Multi-class Classification (3+ classes)	Multiclass Cross-Entropy, KL Divergence Loss

C) Optimizers

Once we know how wrong the model is (the loss), the Optimizer is the tool that actually adjusts the weights and biases to reduce that error.

🕒 Simple analogy: You're standing on a mountain (the loss function's 'landscape'), blindfolded, and you want to walk down to the lowest point (the minimum error). Your feet feel the slope beneath you and help you decide

which direction to step. That's exactly what an optimizer does — it uses the 'slope' (gradient) to decide which direction to adjust the weights.

Gradient Descent (the most important optimizer)

- It calculates how a small change in each weight would affect the loss (this is called the 'gradient').
- It adjusts each weight a small step in the direction that reduces the loss.
- It repeats this over and over until the loss becomes as small as possible.

💡 *The Learning Rate controls how big each step is (usually a small number like 0.1). Too big a step and you might overshoot the best answer (never converge). Too small a step and training takes forever, or you get stuck in a 'local minimum' — a spot that looks like the lowest point but isn't actually the best one overall.*

Other common optimizers (all improve on Gradient Descent)

- Stochastic Gradient Descent (SGD): instead of using ALL data every step, it uses small random batches — much faster, and uses 'momentum' (remembering past steps) to move more smartly.
- Adagrad: gives each individual weight its own learning rate — good for data where many values are missing/sparse. Downside: learning rate can shrink too much over time.
- RMSprop: improves on Adagrad by only remembering gradients from a recent window of time, not forever.
- Adam (Adaptive Moment Estimation): combines ideas of momentum with adaptive learning rates. It is the most widely used optimizer today and is usually a safe default choice.

D) Parameters vs Hyperparameters

	Parameters	Hyperparameters
What is it?	Values the model learns automatically from data	Settings YOU choose manually before training
Examples	Weights, Biases	Learning rate, number of layers, number of epochs, batch size
Can it be estimated from data?	Yes — that's literally how training works	No — you choose it using experience, trial & error, or search methods

💡 *Quick rule of thumb: if YOU have to manually set the value before training starts, it's a hyperparameter. If the model figures it out DURING training, it's a parameter.*

E) Epochs, Batches & Iterations

These terms exist because most real datasets are too big to feed into the computer all at once — so we split them into smaller chunks.

- Epoch: one full pass of the ENTIRE dataset through the network (forward + backward), once.
- Batch Size: how many training examples are used in one single step/pass.
- Iteration: one batch being processed. Number of iterations per epoch = (total data size) ÷ (batch size).

🧠 *Simple analogy: Learning a song's lyrics: reading through the whole song once (1 epoch) isn't enough to memorise it — you need to read it several times (multiple epochs) before you really know it by heart.*

💡 Example: If you have 34,000 examples and use a batch size of 500, you'll need 68 iterations to complete just ONE epoch ($34,000 \div 500 = 68$).

5. Types of Learning

A) Supervised Learning

The model learns from labeled data — meaning every example already comes with the correct answer attached. It's like learning with an answer key.

It has two main types:

- Classification — predicting a category/class. Example: Is this email 'spam' or 'not spam'? Is this a picture of a '3' or a '7'?
- Regression — predicting a continuous number. Example: predicting a house price, or predicting a student's test score based on hours studied.

Common algorithms: Linear Classifiers, Support Vector Machines, Decision Trees, K-Nearest Neighbors, Random Forest (for classification); Linear Regression, Lasso Regression, Multivariate Regression (for regression).

Used in: face/fingerprint recognition, spam detection, speech recognition, medical diagnosis.

B) Unsupervised Learning

The model gets data WITHOUT any labels or correct answers. Its job is to find hidden patterns or groupings on its own.

- Clustering — grouping similar data points together. Example: grouping customers with similar shopping habits. Popular methods: K-Means, Hierarchical Clustering.
- Association — finding relationships between items. Example: 'People who buy potatoes and burgers often also buy beer' (this is how store recommendations / 'frequently bought together' works).

Used in: Netflix/Amazon recommendations, Airbnb matching people to stays, credit card fraud detection (spotting unusual spending patterns).

C) Reinforcement Learning

An 'agent' learns by trial and error inside an environment, getting rewards for good actions and penalties for bad ones — similar to training a pet with treats.

- Environment: the world the agent operates in.
- State: the agent's current situation.
- Reward: feedback received after an action.
- Policy: the strategy the agent uses to decide its next action.

🕹️ Simple analogy: Think of Pac-Man: the agent (Pac-Man) explores a grid (the environment), gets a reward for eating food, and a penalty (losing the game) if a ghost catches it. Over many attempts, it learns the best strategy to win.

Used in: robotics, self-driving car decision-making, game-playing AI, traffic light control.

6. Overfitting & Regularization

A big challenge in deep learning is making a model that works well not just on the data it was trained on, but also on NEW data it has never seen.

Underfitting vs Overfitting

- Underfitting: the model is too simple — it doesn't even learn the training data properly (bad on both training AND test data).
- Overfitting: the model 'memorizes' the training data too closely (including its noise/randomness), so it performs great on training data but poorly on new, unseen data.

🔍 Simple analogy: Imagine memorising the exact answers to last year's exam paper instead of actually understanding the subject. You'd ace that exact paper (training data) but fail a new exam with different questions (test data).

Fixing Overfitting — key techniques

- Get More Data: the best solution when possible — more variety helps the model generalise instead of memorise.
- Dropout: during training, the network randomly 'switches off' some neurons in each round. This stops neurons from becoming too dependent on each other, forcing the network to learn more general, robust patterns.
- Data Augmentation: artificially creating more training examples by slightly modifying existing ones — e.g., flipping, rotating, zooming, or shifting an image. This exposes the model to more variety without needing brand-new data.
- Early Stopping: stop training as soon as performance on the validation set starts getting worse (even if training performance keeps improving) — this is the point where the model starts to overfit.
- Weight Regularization (L1 / L2): add a penalty to the loss function for having very large weights, which keeps the model simpler and less likely to memorise noise. L1 penalises the absolute value of weights; L2 penalises the squared value of weights.

💡 Be careful with data augmentation — don't apply changes that would confuse the correct label. For example, flipping an image of the letter 'b' horizontally can turn it into a 'd', which would teach the model the wrong thing.

7. Neural Network Architectures

A) Fully-Connected Feedforward Neural Networks

The simplest and most basic type. Every neuron in one layer connects to every neuron in the next layer, and information only flows forward (input → hidden layers → output), never backward or in loops.

- More neurons per layer = 'wider' network.
- More layers = 'deeper' network.

💡 Adding more neurons/layers increases the model's power, but also increases computing cost — and this cost grows faster than the number of neurons added, so it's always a trade-off.

Limitation: these networks treat every input independently — they have no memory of past inputs, so they struggle with sequences (like sentences or time-based data).

B) Recurrent Neural Networks (RNNs)

Designed for sequential data — data where order matters, like sentences, speech, or a series of positions over time (e.g. a moving ball).

The key idea is a 'feedback loop': at each step, the network uses BOTH the new input AND information from the previous step. This gives it a kind of short-term memory.

🕒 Simple analogy: Understanding the word 'dog' alone tells you nothing. But 'I have a dog' makes sense because you remember the earlier words in the sentence. RNNs try to do the same — using memory of earlier inputs to understand the current one.

The problem with basic RNNs: Short-Term Memory

As sequences get longer, RNNs struggle to remember information from many steps ago. This happens due to the Vanishing Gradient Problem — during training, the error signal shrinks smaller and smaller as it travels back through many time steps, so earlier steps barely get updated/learned.

Fixing this: Gated RNNs and LSTMs

Type	Number of gates	Gates
Gated RNN	2 gates	Update gate, Reset gate
LSTM (Long Short-Term Memory)	3 gates	Input gate, Output gate, Forget gate

These 'gates' are mechanisms that let the network decide what information to keep, forget, or pass along — allowing it to remember important information over much longer sequences.

RNNs/LSTMs are used in: natural language processing, sentiment analysis, DNA sequence analysis, speech recognition, and language translation.

C) Convolutional Neural Networks (CNNs)

Designed specifically for grid-like data such as images, audio, and video. Inspired by how the visual cortex of the brain processes what we see.

A CNN's hidden layers typically include:

- Convolutional layers: use a small sliding 'filter' (also called a kernel) that scans across the image in small chunks, detecting features like edges or textures.
- Pooling layers: shrink the data down while keeping the most important information. Max Pooling keeps the highest value in a region; Min Pooling keeps the lowest.
- Fully-connected layers: near the end, used to make the final classification decision.

Typical CNN flow for image classification: Input image → Convolution (extract features) → Pooling (shrink/simplify) → repeat this a few times → Fully-connected layers → Output prediction (e.g. 'cat').

CNNs are used heavily in: image recognition, image segmentation, video analysis, and even some natural language processing tasks.

8. The 5 Steps of Every Deep Learning Project

Step 1: Gathering Data

Your model is only as good as the data you feed it — 'bad data = bad model'. Data quality matters just as much as quantity.

- General rule of thumb: you need roughly 10× as much data as the number of parameters in your model.
- For image classification specifically: aim for at least ~1,000 images per class you want to classify.
- Good places to find datasets: UCI Machine Learning Repository, Kaggle, Google Dataset Search, or creating your own using web-scraping tools.

💡 *Reliability matters: check for labeling mistakes and noisy/incorrect data. You'll never remove ALL noise, and that's okay — but too much bad data will hurt your model badly.*

Step 2: Preprocessing the Data

Getting the data ready before training. Key parts of this step:

- Splitting the data into 3 sets: Training set (used to teach the model), Validation set (used to tune the model while training), and Test set (used only at the very end to check real-world performance).
- Formatting: converting data into the format your model needs (e.g., database → CSV).
- Handling Missing Data: either remove incomplete examples, or fill in ('impute') missing values, e.g., using the average value.
- Handling Imbalanced Data: if one class has way more examples than another (e.g., 95% 'not fraud' vs 5% 'fraud'), use techniques like downsampling the majority class and upweighting it, so the model doesn't ignore the rare but important class.
- Feature Scaling: putting all features on a similar scale, using Normalization (rescale values to a 0–1 range) or Standardization (centre data around a mean of 0 with standard deviation of 1). This helps the model learn faster and more reliably.

💡 *Cross-validation is a smart way to reuse your training data for validation — you split the training data multiple different ways and train/validate on each split. K-fold cross-validation is the most popular version of this.*

Step 3: Training the Model

The data is fed into the network, forward propagation produces a prediction, the loss function measures the error, and back propagation adjusts the weights and biases — exactly like we described in Section 3. This is the actual 'learning' phase.

Step 4: Evaluating the Model

After training, the model is tested on the Validation set (data it hasn't directly trained on). This gives an honest idea of how the model would perform on new, real-world data.

Step 5: Optimizing the Model's Accuracy

Based on evaluation results, we fine-tune the model further:

- Adjust hyperparameters: try more epochs, change the learning rate, etc.
- Fix overfitting: get more data, reduce model complexity (fewer parameters), apply weight regularization (L1/L2), apply data augmentation, or use dropout.
- Fix underfitting: increase model complexity (more layers/neurons) so it can capture more detail from the data.

💡 *There's no single 'magic formula' for the perfect model — a lot of this step is trial, error, and experience. You'll get faster at making good choices the more models you build.*

— End of Notes —